ANTHROP\C

Audited with Claude Mythos 5.1

We audited a post-quantum security plugin.

What the model read, how it hunted, how it checked itself, and what it found. Step by step.

18
new findings in code reviewed 11 times

3
proven with real code or dependency source

18
fixed and released as 7.1.0 and 8.1.0

By J Caso | 2026-10-30

github.com/jimcase/capacitor-pq-secure-storage

THE TARGET

A plugin that had already been reviewed 11 times.

`capacitor-pq-secure-storage` keeps post-quantum keys and secrets in phone hardware. ML-DSA signatures, ML-KEM encryption and biometric storage. iOS, Android and web.

~2,900
lines of native code, Swift and Kotlin

~700
lines of TypeScript, bridge and web fallback

45
tests passing before the audit

~50
issues fixed in 11 earlier review rounds

So what is left to find?

THE QUESTION

Who is the attacker?

1. **Hostile JavaScript in the webview**
   Any arguments, any order, in parallel, too big or malformed.

2. **Unlocked phone, no biometrics**
   Tries to read, overwrite, delete or downgrade biometric items.

3. **Backup, root or another app**
   Tries to read items or swap keys for its own.

4. **Crash or re-enrollment mid-operation**
   Half writes, power loss, a new fingerprint. Is the store left broken?

5. **Platform confusion**
   A guarantee on iOS that Android lacks, with no flag to say so.

Out of scope: a jailbreak with a hooked LAContext, full rollback by root, bugs inside the OS or BouncyCastle, and every limit already documented.

THE METHOD

Four steps.

1

Give it context and rules

Source, tests, docs and the history of earlier findings.

2

Hunt with five focus areas

iOS, Android, bridge, crypto and a regression pass.

3

Make it kill its own findings

A second pass that tries to prove each one wrong.

4

Run the claims that can be run

Small experiments, with output kept as proof.

> **All of it ran inside Claude Code, with the repository and a shell at hand.**

STEP 1

Context and strict rules.

What it read

Swift and Kotlin sources in full. The TypeScript layer and the web fallback. Tests, docs, CI and build files. The July catalog of ~50 earlier findings.

Rules for every finding

1. File and line, with the code quoted.
2. A concrete attack: who, in what order, what they get.
3. The fix in one or two sentences.
4. Severity and an honest confidence.
5. No style notes. No limits that are already documented.

> **The rule: prefer five findings you are sure of over twenty you are not.**

STEP 2

Five lenses, not one big question.

iOS	Keychain access control, Secure Enclave use, LAContext, atomic writes, error paths.
Android	Keystore parameters, BiometricPrompt binding, AES-GCM at rest, BouncyCastle ML-KEM.
Bridge	The TypeScript layer, argument checks, what the web fallback promises, hostile JS.
Crypto	Keys, nonces, encodings, signature inputs. Do the three platforms agree?
Regression	Walk the old catalog and check that every fix is still in the code.

The crypto lens found the ML-KEM mismatch. The regression lens found fixes that never worked.

STEP 3

Try to kill every finding.

A second pass reads the code around each finding, traces every path that reaches it, then decides.

CONFIRMED The scenario works as described.

PLAUSIBLE Likely real, but it depends on device behavior.

REJECTED Not reachable, already mitigated, or a known limit.

About a dozen suspects were dropped

A leak in the auth latch: every path traced, no leak.
A key tag swap: needs root, same user.
The tier probe on a locked iPhone: fails closed.
ECDSA low-S: iOS, Android and noble agree.
The web fallback: nothing new.

STEP 4

Run what can be run.

BouncyCastle against noble

ML-KEM keygen in BouncyCastle 1.85. Encapsulate with noble 0.6.1, the version the plugin pins. Decapsulate with the plugin's own call. **The secrets do not match.** With withNoKdf() they do.

One line on the JVM

Signature.getInstance("EC") on JDK 24 throws NoSuchAlgorithmException. The ECDSA overwrite gate can never open.

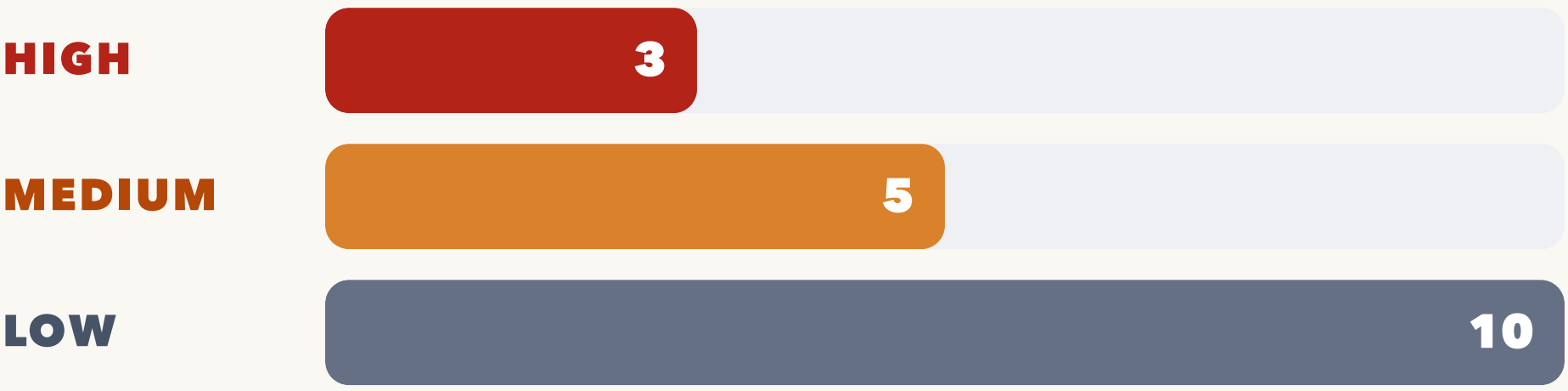
Read the dependency

Capacitor 8.4.1 rethrows plugin exceptions as RuntimeException. The bc-java source shows the default KDF is KDF3 with SHA-256.

> **A claim with output beats a claim with a quote.**

RESULTS

18 new findings.



One of the three high findings, M-8, is only plausible until a real iPhone confirms it.

3
proven: two executed, one checked in dependency source

8
entries of the old catalog were incomplete or wrong

8
device checks still open

18
fixed and released in 7.1.0 and 8.1.0

Two of the three proven findings are regressions: a fix that never worked, and a fix that opened a crash.

FINDING M-3 | MEDIUM | ANDROID

Same key exchange, different secret.

Android used BouncyCastle ML-KEM with its defaults. The defaults pass the shared secret through KDF3-SHA256. iOS and web use the raw FIPS 203 secret. Data encrypted on one platform does not open on the other.

```
plugin call:
  f46b31886a143eb750e828d8c833c32d
  8c9b9d59375cb0b2a929a8539a768173
noble (raw K):
  94b8608b73b00f35c99d0f5daabcdc7c
  ca0fd7fa4e207d64c41424bdd4afda7b
MATCH_DEFAULT=false   MATCH_NOKDF=true
```

Why the tests missed it: the unit test compared BouncyCastle with BouncyCastle.

Fix: Builder(...).withNoKdf() plus a known-answer test with vectors from another library. Shipped in 7.1.0 and 8.1.0.

FINDING M-4 | MEDIUM | ANDROID

One bad string closes the app.

Base64 decoding sits outside the try block in sign, encryptAtRest and encryptTo.

```
// sign(), line 494
val input = decodeCapped(call, data) ?: return
// decodeCapped(), line 261
val bytes = Base64.decode(data, Base64.DEFAULT)
```

The Capacitor bridge then does this:

```
} catch (Exception ex) {
    Logger.error("Serious error executing plugin", ex);
    throw new RuntimeException(ex);
}
```

Attack: sign({ data: "***" }) from the webview. The process dies. It is a regression of an earlier fix (E-11).

Fix: catch IllegalArgumentException inside decodeCapped(). Shipped in 7.1.0 and 8.1.0.

FINDING M-5 | MEDIUM | ANDROID

The gate that never opens for EC keys.

To overwrite a biometric key, the plugin asks for a biometric bound to that key. It builds the signer from the key algorithm name.

```
val signer = Signature.getInstance(priv.algorithm)
```

For an EC key that name is "EC". It is a KeyFactory name, not a Signature name:

```
NoSuchAlgorithmException:
EC Signature not available
```

Result: overwriting a biometric ECDSA key always fails. There is no delete API, so the alias is lost for good. Earlier fixes A-1 and A-2 never worked for EC.

Fix: map EC to SHA256withECDSA. Shipped in 7.1.0 and 8.1.0.

FINDINGS M-1 AND M-2 | HIGH | ANDROID

Two ways around the biometric gate.

M-1: a biometric item under a silent key

A failed silent write leaves a silent key behind. The next biometric setItem reuses it. The user sees a prompt and trusts it. From then on the item reads with no prompt.

Fix: only reuse a key that has a real item behind it.

M-2: clear() wipes with no prompt

After a biometric re-enrollment all old keys are dead. If clear() picks a dead one for its prompt, the exception path wipes everything, live items included.

Fix: bind the prompt to a key that is still alive.

Both confirmed by reading the code and fixed in 7.1.0 and 8.1.0. The trigger for each still needs a real device.

ALL FINDINGS | 1 OF 2

M-1 to M-9

Plugin.kt is the Android plugin. Plugin.swift is the iOS plugin. HIGH? means high if confirmed. The chip after each location is how the fix was checked.

M-1	HIGH	setItem reuses an orphan key with the wrong tier	`Plugin.kt:1212–1227` TESTED
M-2	HIGH	clear() wipes with no prompt when the first biometric key is invalidated	`Plugin.kt:1407–1435` TESTED
M-3	MEDIUM	ML-KEM secret on Android is not the raw K	`Plugin.kt:1275, 1299` TESTED
M-4	MEDIUM	Invalid base64 crashes the host app	`Plugin.kt:494, 539, 745` TESTED
M-5	MEDIUM	Overwrite gate fails for ECDSA keys, alias is bricked	`Plugin.kt:430–453` TESTED
M-6	MEDIUM	Ed25519 wrap key is retired before the commit	`Plugin.kt:108–188` BUILT
M-7	MEDIUM	One alias can hold two keys	`Plugin.kt:111, 308, 461` BUILT
M-8	HIGH?	clear() and keys() enumerate with UISkip	`Plugin.swift:1362–1369, 1400–1411` DEVICE
M-9	LOW	biometricGated ignores enrollment	`Plugin.swift:263–267` TESTED

ALL FINDINGS | 2 OF 2

M-10 to M-18

All low severity. Hardening, docs and small hostile-input cases. The chip after each location is how the fix was checked.

M-10 LOW — Silent keys usable while the device is locked
`Plugin.kt:405–416, 955–995` BUILT

M-11 LOW — sign() prompts before validating
`Plugin.swift:411–422` TESTED

M-12 LOW — setItem limits count characters, not bytes
`Plugin.swift:1159–1161` TESTED

M-13 LOW — recipientPublicKey has no cap before decoding
`Plugin.swift:539–543` TESTED

M-14 LOW — Secure Enclave signing key is created without a lock
`Plugin.swift:757–786` BUILT

M-15 LOW — Backup exclusion in the README is incomplete
`README.md:241–259` DOCS

M-16 LOW — Key prefix is global state
`enhance.ts:20–37` TESTED

M-17 LOW — Caller controls the prompt title on Ed25519
`Plugin.kt:216, 1049–1053` BUILT

M-18 LOW — Non-boolean flags are treated as defaults
`Plugin.kt:1205, Plugin.swift:1177` TESTED

HONEST LIMITS

What it could not do.

No real device during the audit, so no Secure Enclave and no KeyMint. The fixes were checked on a simulator and an emulator. M-8 is a guess until a phone answers. **Eight device checks are still open.**

AFTER THE AUDIT

1. One commit per finding, red test first.

2. Shipped as 7.1.0 and 8.1.0.

3. 52 JS, 30 JVM and 15 iOS tests pass.

4. Old Android ML-KEM frames no longer open. They never worked across platforms.

PLAYBOOK

Run it on your code.

1. **Pin the code**
   One commit, clean tree. Write the hash in the report.
2. **Write the threat model first**
   Who attacks, what counts, what does not count.
3. **Give it everything**
   Source, tests, docs, CI and the history of earlier findings.
4. **Set strict rules**
   File and line, a concrete scenario, a fix, severity, honest confidence.
5. **Hunt with several lenses**
   One focus per pass: each lens finds different bugs.
6. **Make it kill its findings**
   Second pass with three verdicts: confirmed, plausible, rejected.
7. **Run what can be run**
   Small experiments on the JVM or in Node. Keep the output.
8. **Write down the gaps**
   What was not covered and what needs a real device.
9. **Fix with a red test first**
   One commit per finding.

ANTHROP\C

Audited with Claude Mythos 5.1.

18 findings with file, line, scenario and fix. Two proven by running code. All 18 fixed in 7.1.0 and 8.1.0. The full report has every detail.

github.com/jimcase/capacitor-pq-secure-storage

Independent audit of the author's own open-source plugin, run with Claude Mythos 5.1. Claude, Mythos and the Anthropic logo are trademarks of Anthropic, PBC. This document is not affiliated with, sponsored or endorsed by Anthropic.